

前端面试通关宝典

爪哇教育

方法论.....	2
Javascript	2
Javascript 有多少种数据类型，如何判断？	2
为什么说 Symbol 是基本数据类型？	2
如何理解原型链？	2
如何理解闭包？	3
暂时性死区.....	3
在 try-catch 或 with 中定义变量有什么影响？	4
手写专题	5
实现继承	5
手写 apply, call, bind.....	6
手写实现 new 操作符的能力	8
手写实现 instanceof.....	8
手写实现 Promise	8
手写 Promise.prototype.catch; Promise.prototype.finally; Promise.all.....	9
手写防抖节流.....	10
实现柯里化.....	12
实现发布订阅模式；观察者模式	13
限流调度器.....	15
HTML5	16
CSS.....	17
DOM	17
框架.....	18
Vue	18
HTTP	18
工程能力	20
Web 安全	20
跨域.....	20
点击劫持 Click Jacking.....	21
中间人攻击.....	21
Typescript.....	22
计算机网络.....	22
CDN，回源.....	22
算法修炼	22
常见数据结构.....	22
技巧	23
综合题	23
项目亮点总结.....	23

方法论

- 要学习一种方案或者技术，不妨找一找相关的最流行的开源项目，学学它的实现。
- 自己动手总结，定期复习，强化记忆。

Javascript

Javascript 有多少种数据类型，如何判断？

按大的类型来说。

primitive types (基本类型，栈内存): Number String Boolean Null Undefined Symbol object types (对象类型，堆内存): Object Array Date RegExp Map Set 等等

结合 `typeof` 和 `instanceof` 可以判断，但是比较繁琐

一个通用的方法是通过 `Object.prototype.toString.call` 来判断。

```
JavaScript 复制代码
1 function getType(val) {
2   return Object.prototype.toString.call(val).replace(/\[object\s(\w+)\]/,
3     .toLowerCase());
4 }
```

为什么说 Symbol 是基本数据类型？

Symbol 是没有构造函数 `constructor` 的，不能通过 `new Symbol()` 获得实例。

基本类型为什么也可以调方法，比如 `toFixed()` ？

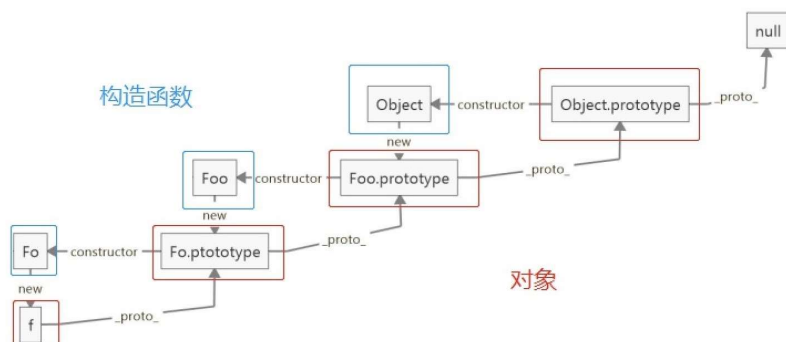
基本类型都有对应的包装类，可以调方法是因为进行了自动装箱。

如何理解原型链？

思路：首先要说什么是原型，为什么要设计原型（共享属性和方法），然后说属性和方法查找的顺序，自然而然就谈到了原型链。原型链可以引申到继承，继承要结合构造函数和原型。

每个对象都有原型，对象的原型可以通过其构造函数的 `prototype` 属性访问。查找一个对象的属性或方法时，如果在对象本身上没有，就会去其原型上查找；而原型本身也是一个对象，如果在

原型上也找不到，就会继续找原型的原型，从而串起一个原型链，原型链的终点是 `null`。



继承后面再写专题！

如何理解闭包？

思路：闭包由词法环境和函数组成

内部函数 inner 如果引用了外部函数 outer 的变量 a，会形成闭包。如果这个内部函数作为外部函数的返回值，就会形成词法环境的引用闭环（循环引用），对应变量 a 就会常驻在内存中，形成大家所说的“闭包内存泄漏”。虽然闭包有内存上的问题，但是却突破了函数作用域的限制，使函数内外搭起了沟通的桥梁。闭包也是实现私有方法或属性，暴露部分公共方法的渠道。还可以引申出柯里化，bind 等概念，面试官说“可以啊，小伙子！”

变量提升问题

JS 分为词法分析阶段和代码执行阶段。在词法分析阶段，变量和函数会被提升（Hoist），注意 let 和 const 不会提升。而赋值操作以及函数表达式是不会被提升的。

出现变量声明和函数声明重名的情况时，函数声明优先级高，会覆盖掉同名的变量声明！为了降低出错的风险，尽量避免使用 hoist！

注意混杂了函数参数时的坑，主要还是考虑按两个阶段分析，包括函数参数也是有声明和赋值的阶段：

```
JavaScript 复制代码
1 var obj = {
2   val: 5
3 }
4 function test(obj) {
5   obj.val = 10;
6   console.log(obj.val);
7   var obj = { val: 20 };
8   console.log(obj.val)
9 }
10 test(obj);
11 console.log(obj.val)
12
13 // 输出 10 20 10
```

```
JavaScript 复制代码
1 function test(obj) {
2   console.log(typeof obj)
3   function obj() {}
4 }
5 test({})
6
7 // 输出 function
```

暂时性死区

这个其实在 ES6 规范就有提到了：The variables are created when their containing [Lexical Environment](https://262.ecma-international.org/6.0/#sec-lexical-environments) is instantiated but may not be accessed in any way until the variable's LexicalBinding is evaluated.

通过 let 或 const 声明的变量是在包围它的词法环境实例化时被创建，但是在变量的词法绑定执行前，该变量不能被访问。

```
JavaScript 复制代码
1 typeof a; // 会报错: Uncaught ReferenceError: a is not defined
2 let a;
```

var 是没有这个问题的，var 声明变量有变量提升，并且变量的初始值是 undefined。

警告：养成先声明再使用的好习惯，别花里胡哨的！

诡异的块级函数声明

```
1 a = 3
2 {
3   a = 2
4   function a() {}
5   a = 1
6 }
7 console.log(a)
8 // 输出 2
```

严格模式要注意的地方

- Module code is always strict mode code.
- All parts of a ClassDeclaration or a ClassExpression are strict mode code.
- 禁止意外创建全局变量，直接抛出错误
- this 问题
- arguments 和参数列表的独立性，并且 arguments 不可修改
- 给 NaN 赋值会抛出错误
- 对象操作：给不可写属性赋值(writable: false)；给只读属性赋值(只有 get，没有 set)；给不可扩展对象新增属性
- 严格模式要求参数名唯一，属性名唯一
- 禁止八进制数字
- 禁止对 primitive type 的变量加属性或方法
- 禁止使用 with
- eval 不再为 surrounding scope 添加新的变量
- 禁止 delete 声明的变量，禁止 delete 不可删除的属性或方法

LHS 和 RHS 是什么？会造成什么影响？

LHS 是 Left Hand Side 的意思，左值查询一个变量，如果变量不存在，且在非严格模式下，就会创建一个全局变量。RHS 查询一个变量，如果变量不存在，就会报错 ReferenceError。

常见的 JS 语法错误类型有哪些？举几个例子。

ReferenceError: 引用了不存在的变量

TypeError: 类型错误，比如对数值类型调用了方法。

SyntaxError: 语法错误，词法分析阶段报错。

RangeError: 数值越界。

URIError, EvalError

在 try-catch 或 with 中定义变量有什么影响？

catch子句和 with 语句中有自己的词法作用域，但是如果通过 var 定义变量，会直接影响 try-catch 或 with 语句所在词法作用域。

JS 中如何确定 this 的值？如果是在全局作用域下，this 的值是全局对象，浏览器环境下是 window。



函数中 this 的指向取决于函数的调用形式，在一些情况下也受到严格模式的影响。

- 作为普通函数调用：严格模式下，this 的值是 undefined，非严格模式下，this 指向全局对象。
- 作为方法调用：this 指向所属对象。
- 作为构造函数调用：this 指向实例化的对象。
- 通过 call, apply, bind 调用：如果指定了第一个参数 thisArg，this 的值就是 thisArg 的值（如果是原始值，会包装为对象）；如果不传 thisArg，要判断严格模式，严格模式下 this 是 undefined，非严格模式下 this 指向全局对象。

手写专题

实现继承

说出中心思想，而不是列举被博客炒了一遍又一遍的冷饭。

实现继承有两个方面要考虑，一个是原型属性和方法的继承，另一个是构造器的继承。

```
1 * function inherit(ChildClass, SuperClass) {
2   ChildClass.prototype = Object.create(SuperClass.prototype)
3   ChildClass.prototype.constructor = ChildClass;
4 }
5
6 * function SuperClass() {
7   this.propA = 'a'
8 }
9
10 * function ChildClass() {
11   SuperClass.call(this)
12   this.propB = 'b'
13 }
14
15 inherit(ChildClass, SuperClass)
16
17 * function A(name) {
18   this.name = name;
19 }
20 * A.prototype.getName = function () {
21   console.log(this.name);
22 };
23
24
25 //es5写法
26 * function B(){
27   A.apply(this,arguments);
28 }
29 const _proto_ = Object.create(A.prototype);
30 _proto_.constructor = B;
31 B.prototype = _proto_;
32 const b = new B('lisa');
33 b.getName();
34
35 //es6写法
36 * class A {
37 *   constructor(name) {
38     this.name = name;
39   }
40 *   getName() {
41     console.log(this.name);
42   }
43 }
44 * class B extends A {
45 *   constructor(name) {
46     super(name);
47   }
48 }
49 const b = new B('lisa');
50 console.log(b.name);
```

手写 apply, call, bind

call 和 apply 是用来绑定 this，并提供参数，执行后会立即运行原函数。call 以列表形式提供参数，apply 以数组形式提供传参。

- 手写 call


```
1 // 如果不能用解构, arguments可以这样转数组
2 * function args2Array(args) {
3   var arr = [];
4   for (var i = 0; i < args.length; i++) {
5     arr[i] = args[i]
6   }
7   return arr;
8 }
9
10 * function getGlobal() {
11   return (function(){ return this; })()
12 }
13
14 * Array.prototype.myCall = function() {
15   var thisArg = arguments[0]
16   var args = [...arguments].slice(1);
17   var invokeFunc = this;
18   var isStrict = (function(){return this === undefined})();
19   if (isStrict) {
20     if (typeof thisArg === 'number') {
21       thisArg = new Number(thisArg)
22     } else if (typeof thisArg === 'string') {
23       thisArg = new String(thisArg)
24     } else if (typeof thisArg === 'boolean') {
25       thisArg = new Boolean(thisArg)
26     }
27   }
28
29   if (!thisArg) {
30     return invokeFunc(...args)
31   }
32   var uniqProp = Symbol()
33   thisArg[uniqProp] = invokeFunc;
34   return thisArg[uniqProp](...args)
35 }
```

- 手写apply

```
1 * Array.prototype.myApply = function(thisArg, args) {
2   var invokeFunc = this;
3   var isStrict = (function(){return this === undefined})();
4   if (isStrict) {
5     if (typeof thisArg === 'number') {
6       thisArg = new Number(thisArg)
7     } else if (typeof thisArg === 'string') {
8       thisArg = new String(thisArg)
9     } else if (typeof thisArg === 'boolean') {
10      thisArg = new Boolean(thisArg)
11    }
12  }
13
14  if (!thisArg) {
15    return invokeFunc(...args)
16  }
17  const uniqProp = Symbol()
18  thisArg[uniqProp] = invokeFunc;
19  return thisArg[uniqProp](...args)
20 }
```

- 手写bind

JavaScript 复制代码

```
1 * Function.prototype.myBind = function() {
2   const thisArg = arguments[0]
3   const boundParams = [...arguments].slice(1)
4   const boundTargetFunc = this;
5   * if (typeof boundTargetFunc !== 'function') {
6     throw new TypeError('the bound target function must be a function.
7   }
8   * function fBound() {
9     const restParams = [...arguments]
10    const allParams = boundParams.concat(restParams)
11    return boundTargetFunc.apply(this instanceof fBound ? this : thisA
12  }
13  fBound.prototype = Object.create(boundTargetFunc.prototype)
14  return fBound;
15 }
```

手写实现 new 操作符的能力

手写实现new，只能用函数的形式封装。首先要搞清楚new的过程

1. 创建一个新对象obj
2. 新对象obj的原型指向Constructor.prototype(2, 3两步可以通过Object.create(Constructor.prototype))
3. 执行构造函数函数体的内容
4. 返回这个对象

JavaScript 复制代码

```
1 * function myNew(Constructor, ...args) {
2   var obj = Object.create(Constructor.prototype)
3   Constructor.call(obj, ...args)
4   return obj;
5 }
```

手写实现 instanceof

instanceof判断的是右操作数的prototype属性是否出现在左操作数的原型链上。核心是要拿到左操作数的原型进行检查，要顺着原型链检查。取得原型是利用了Object.getPrototypeOf(obj)。

JavaScript 复制代码

```
1 * function myInstanceof(obj, ctor) {
2   * if (typeof ctor !== "function") {
3     throw new TypeError("the second paramater must be a function")
4   }
5
6   const rightProto = ctor.prototype;
7
8   let leftProto = Object.getPrototypeOf(obj);
9
10  let isInstanceFlag = leftProto === rightProto;
11
12  * while (!isInstanceFlag && leftProto) {
13    * if (leftProto === rightProto) {
14      isInstanceFlag = true;
15    } else {
16      leftProto = Object.getPrototypeOf(leftProto);
17    }
18  }
19
20  return isInstanceFlag;
21 }
```

手写实现 Promise

这个一般不会直接出现吧，因为如果按 Promise/A+规范来，代码量不少，如果做题时能提供

<<https://github.com/cumt-robin/promises-aplus-robin>>。

手写 Promise.prototype.catch

catch是基于 `Promise.prototype.then` 实现的，所以就有点简单了。

```
JavaScript 复制代码
1 Promise.prototype.myCatch = function(onRejected) {
2   return this.then(undefined, onRejected)
3 }
```

手写 Promise.prototype.finally

这个是有可能会考的，比如微信小程序就不支持 finally。可以基于 `.then` 来实现，不管 fulfilled 还是 rejected 都要执行onFinally。

但是要注意，不管当前Promise的状态是 fulfilled 还是 rejected，只要在 onFinally 中没有发生以下任何一条情况，finally 方法返回的新的 Promise 实例的状态就会与当前 Promise 的状态保持一致！这也意味着即使在 onFinally 中返回一个状态为 fulfilled 的 Promise 也不能阻止新的 Promise 实例采纳当前 Promise 的状态或值！

- 返回一个状态为或将为 rejected 的Promise
- 抛出错误

总的来说，在 finally 情况下，rejected 优先！

```
JavaScript 复制代码
1 Promise.prototype.myFinally = function(onFinally) {
2   return this.then(
3     value => {
4       return Promise.resolve(onFinally()).then(() => value)
5     },
6     reason => {
7       return Promise.resolve(onFinally()).then(() => { throw reason })
8     }
9   );
10 }
```

手写 Promise.all

这个主要是考察如何收集每一个 Promise 的状态变化，在最后一个 Promise 状态变化时，对外发出信号。

- 判断 iterable 是否空
- 判断 iterable 是否全部不是 Promise
- 遍历，如果某项是 Promise，利用 `.then` 获取结果，如果 fulfilled，将 value 存在 values 中，并用 fulfillCount 计数；如果是 rejected，直接 reject reason。
- 如果某项不是 Promise，直接将值存起来，并计数。
- 等所有异步都 fulfilled，fulfillCount 必将是 iterable 的长度（在 onFulfilled 中判断 fulfillCount），此时可以 resolve values。

```
1 * Promise.all = function(iterable) {
2   var tasks = Array.from(iterable)
3   * if (tasks.length === 0) {
4     return Promise.resolve([]);
5   }
6   * if (tasks.every(task => !(task instanceof Promise))) {
7     return Promise.resolve(tasks);
8   }
9   * return new Promise((resolve, reject) => {
10    var values = new Array(tasks.length).fill(null);
11    var fulfillCount = 0;
12    * tasks.forEach((task, index, arr) => {
13      * if (task instanceof Promise) {
14        * task.then(value => {
15          fulfillCount++;
16          values[index] = value;
17        * if (fulfillCount === arr.length) {
18          resolve(values)
19        }
20        * }, reason => {
21          reject(reason)
22        * })
23      * } else {
24        fulfillCount++;
25        values[index] = task;
26      }
27    * })
28  * })
29 }
```

手写防抖节流

防抖

原理：

防抖 (debounce)：不管事件触发频率多高，一定在事件触发 n 秒后才执行，如果你在一个事件触发的 n 秒内又触发了这个事件，就以新的事件的时间为准，n 秒后才执行，总之，触发完事件 n 秒内不再触发事件，n 秒后再执行。(频繁触发就执行最后一次)

应用场景：

1. 窗口大小变化，调整样式
2. 搜索框，输入后 1000 毫秒搜索
3. 表单验证，输入1000 毫秒后验证
4. 频繁点击按钮，使用防抖避免重复提交请求

防抖实现：

- debounce 实则是个包装函数，通过传入操作函数和时间间隔，来返回一个新函数
- 新函数中主要是通过定时器来设置函数调用的频率
- flag 只有第一次触发的时候会立即执行

```
1 //flag是否立即执行
2 * function debounce(handler,ms,flag){
3     let timer = null;
4     * return function(...args){
5         clearTimeout(timer);
6     * if(flag&&!timer){
7         handler.apply(this,args);
8     }
9     * timer = setTimeout(()=>{
10         handler.apply(this,args);
11     },ms)
12 }
13 }
14 //demo
15 window.addEventListener('resize',debounce(handler,1000));
16 * function handler(){
17     console.log('ok');
18 }
```

节流

原理:

节流 (throttle) :不管事件触发频率多高, 只在单位时间内执行一次。(频繁触发, 还是按照时间间隔执行)

应用场景:

1. 鼠标不断点击触发, mousedown(单位时间内只触发一次)
2. 监听滚动事件, 比如是否滑到底部自动加载更多, 用throttle 来判断

节流实现

- 和防抖不同的是, 防抖中是取消定时器, 节流中是定时器到时间自动执行, 仅仅是将 timer 变量设置为 null
- 时间戳版: 第一次执行, 最后一次不执行
- 定时器版: 第一次不执行, 最后一次执行

```
1 //时间戳版
2 * function throttle(handler,ms){
3     let pre = 0;
4     * return function(...args){
5     * if(Date.now()-pre > ms){
6         pre = Date.now();
7         handler.apply(this,args);
8     }
9 }
10 }
11 //定时器版
12 * function throttle(handler,ms){
13     let timer = null;
14     * return function(...args){
15     * if(!timer){
16     * timer = setTimeout(()=>{
17         timer = null;
18         handler.apply(this,args);
19     },ms)
20 }
21 }
22 }
23 //demo
24 document.getElementById('btn').addEventListener('click', throttle1(handler));
25 * function handler() {
26     console.log('ok');
27 }
```

考虑多种数据类型的处理

```
JavaScript 复制代码
1 * function deepClone(val) {
2   var type = getType(val)
3 *   if (type === 'object') {
4     var result = {};
5 *     Object.keys(val).forEach(key => {
6       result[key] = deepClone(val[key])
7     })
8 *   } else if (type === 'array') {
9     return val.map(item => deepClone(item))
10 * } else if (type === 'date') {
11   return new Date(val.getTime())
12 * } else if (type === 'regexp') {
13   return new RegExp(val.source, val.flags)
14 * } else if (type === 'function') {
15   return eval("(" + val.toString() + ")")
16 * } else if (type === 'map' || type === 'set') {
17   return new val.constructor(val)
18 * } else {
19   return val;
20 }
21 }
```

实现柯里化

柯里化是把接受多个参数的函数变换成接受一个单一参数(最初函数的第一个参数)的函数，并且返回接受余下的参数且返回结果的新函数的技术

柯里化某种意义上就是预置参数。比如：

```
JavaScript 复制代码
1 function add(a, b) { return a + b; }
2 var add1 = function(val) { return add(1 + val) }
```

柯里化有延迟计算的作用，参数的缓存是通过闭包实现的，所以实现上可以是：

```
1 // 按定义实现
2 * function curry(fn, presetParam) {
3 *   return function() {
4 *     return fn.apply(this, [presetParam, ...arguments])
5 *   }
6 * }
7
8 // 扩展多参数
9 * function curry(fn, ...args) {
10 *   return function() {
11 *     return fn.apply(this, [...args, ...arguments])
12 *   }
13 * }
14
15 // 定长柯里化, 执行时机判断
16 * function curry(fn, ...args) {
17 *   var len = fn.length;
18 *   return function() {
19 *     var allArgs = [...args, ...arguments]
20 *     if (allArgs.length >= len) {
21 *       return fn.apply(this, allArgs)
22 *     } else {
23 *       return curry.call(null, fn.bind(this), ...allArgs)
24 *     }
25 *   }
26 * }
27
28 // 不定长柯里化
```

实现发布订阅模式

发布订阅模式本质上是实现一个事件总线。只要实现其核心的 on, emit, off, remove, once 等方法即可。




```
1 * class Listener {
2 *   constructor(id, eventName, callback) {
3 *     this.id = id;
4 *     this.eventName = eventName;
5 *     this.callback = callback;
6 *   }
7 * }
8
9 * class EventBus {
10 *   constructor() {
11 *     this.events = {};
12 *     this.autoIncreaseId = 1;
13 *   }
14
15 *   addListener(listener) {
16 *     if (this.events[listener.eventName]) {
17 *       this.events[listener.eventName].push(listener)
18 *     } else {
19 *       this.events[listener.eventName] = [listener]
20 *     }
21 *     return listener;
22 *   }
23
24 *   on(eventName, handler) {
25 *     const listener = new Listener(this.autoIncreaseId++, eventName, handler);
26 *     return this.addListener(listener)
27 *   }
28
29 *   emit(eventName, ...args) {
30 *     if (this.events[eventName]) {
31 *       this.events[eventName].forEach(({ callback }) => {
32 *         callback(...args);
33 *       })
34 *     }
35 *   }
36
37 *   off(eventName, listener) {
38 *     const listeners = this.events[eventName]
39 *     if (listeners) {
40 *       const index = listeners.findIndex(l => l.id === listener.id)
41 *       if (index !== -1) {
42 *         listeners.splice(index, 1)
43 *       }
44 *     }
45 *   }
46
47 *   remove(eventName) {
48 *     if (this.events[eventName]) {
49 *       delete this.events[eventName]
50 *     }
51 *   }
52
53 *   once(eventName, handler) {
54 *     // 需要搞一个listener的概念
55 *     const that = this;
56 *     const id = this.autoIncreaseId++
57 *     const onceCallback = function() {
58 *       handler(...arguments);
59 *       const index = that.events[eventName].findIndex(l => l.id === id)
60 *       that.events[eventName].splice(index, 1)
61 *     }
62 *     const listener = new Listener(id, eventName, onceCallback)
63 *     return this.addListener(listener)
64 *   }
65 }
```


实现观察者模式

很长一段时间，我都在纠结发布订阅和观察者模式的区别，而没有找到比较让我满意的答案。甚至在我自己手动实现了两种模式的代码后，我还时不时地产生模糊，因为它们都有涉及事件和回调的概念。终于，我认清了，发布订阅模式是一个多事件的事件总线；而观察者模式是针对单主题的，这是二者最大的区别。

举个例子，发布订阅模式就是，我请了一个私家侦探，要求监视目标的一举一动；而观察者模式就是，我只关注目标有没有和另一个人在一起，哈哈。

由于观察者模式界限明确，有主题和观察者两部分，所以分别用Subject和Observer类来实现。

```
JavaScript | 复制代码

1 * class Observer {
2   static autoIncreaseId = 1
3
4   constructor(callback) {
5     this.id = Observer.autoIncreaseId++;
6     this.callback = callback
7   }
8
9   // 暴露一个接口，让主题去调用，当然命名应该是约定好的
10  * complete(...args) {
11    this.callback(...args);
12  }
13 }
14
15 class Subject {
16   constructor(name) {
17     this.name = name;
18     this.observerList = [];
19   }
20
21   addObserver(observer) {
22     this.observerList.push(observer)
23   }
24
25   removeObserver(observer) {
26     const index = this.observerList.findIndex(item => item.id === ...);
27     if (index !== -1) {
28       this.observerList.splice(index, 1)
29     }
30   }
31
32   * notify(...args) {
33     this.observerList.forEach(observer => {
34       observer.complete(...args);
35     })
36   }
37 }
```

限流调度器

进一个任务，taskCount+1，等 Promise finally 后，才 taskCount-1；如果 taskCount 达到 taskCountLimit 上限，不能进任务。

```
1 class Scheduler {
2   constructor(limit) {
3     this.taskCountLimit = limit;
4     this.taskCount = 0;
5   }
6
7   enter(taskGenerator) {
8     if (this.taskCount === this.taskCountLimit) {
9       throw new Error(`The number of concurrent tasks cannot exceed`);
10    } else {
11      this.taskCount++;
12      taskGenerator().finally(() => {
13        this.taskCount--;
14      });
15    }
16  }
17 }
18
19 // 测试代码
20 const s = new Scheduler(3);
21
22 // 进第1个任务
23 s.enter(
24   () =>
25     new Promise((resolve) => {
26       setTimeout(() => {
27         resolve(1);
28       }, 3000)
29     })
30 );
31
32 // 进第2个任务
33 s.enter(
34   () =>
35     new Promise((resolve) => {
36       setTimeout(() => {
37         resolve(2);
38       }, 5000)
39     })
40 );
41
42 // 进第3个任务
43 s.enter(
44   () =>
45     new Promise((resolve) => {
46       setTimeout(() => {
47         resolve(3);
48       }, 8000)
49     })
50 );
51
52 // 进第4个任务，抛出异常
53 s.enter(
54   () =>
55     new Promise((resolve) => {
56       setTimeout(() => {
57         resolve(4);
58       }, 1000)
59     })
60 );
```

HTML5

sessionStorage 和 localStorage 的细节

无痕模式会有一些影响，以在 Chrome 测试得出的结论为准。

sessionStorage

- 基于会话级的存储，浏览器会话结束时，sessionStorage 会被清除。
- 刷新页面不会重置 sessionStorage。
- 由一个父窗口打开的新窗口会共享同一个 sessionStorage。
- 手动通过 URL 打开各自的标签页，sessionStorage 不共享。
- 受同源策略限制。
- 无痕模式下，从父窗口打开同源新窗口也无法共享 sessionStorage。

localStorage

- 除非用户手动清除，localStorage 永远不会自动失效。
- 受同源策略限制。
- 浏览器无痕浏览模式下，localStorage 会在私密窗口下最后一个标签页关闭时清除。

CSS

如何理解浮动？和绝对定位的区别是什么？

什么是 BFC？触发 BFC 的条件是？常见应用是啥？

- Block Format Context，块级格式化上下文
- 一块独立的渲染区域，内部元素的渲染不会影响边界以外的元素

触发 BFC 的条件是：

- float 不是 none
- position 是 absolute 或 fixed
- overflow 不是 visible
- display 是 flex、inline-block 等常见应用：
- 清除浮动

什么是层叠上下文？

DOM

移动端 300ms 延迟是指什么？

早期的网站都是为 PC 端而设计，而在移动端访问就会显得大小不合适。iPhone 发布之前，就针对这个问题提出了双击缩放的概念。但是这就带来了一个问题，如何判断一个用户到底是想点击一个元素，还是想双击缩放呢？早期移动端浏览器就只能在 touchend 事件之后再等待 300ms，判断用户是不是会再次点击，如果用户在 300ms 内不会再次点击屏幕就触发 click 事件。但是这样就会造成 300ms 的延迟。

点击穿透问题？

Vue

数据流问题 响应式原理 双向绑定原理 依赖收集的过程

Patch 过程 nextTick 原理

watch 和 computed 原理

computed 是怎么收集依赖的?

watchEffect 是怎么收集依赖的?

React

HTTP

cookie

HTTP 协议本身是无状态的，而 Cookie 是在 HTTP 中的一个请求头，用来保存一小块数据，Cookie 会传递到服务端，用来作为会话标识。服务端可以通过 Set-Cookie 来设置 cookie，而客户端网页 也可以通过 document.cookie 来操作 cookie。

document 操作 cookie

document.cookie 是可以读取所有 cookie 的，最后的值是类似这样的一段字符串。

```
JavaScript 复制代码
1 document.cookie;
2 "remember=1; token=03gzSC2uNLeH00%2B%2Frvcrrj9Xl%2F2lze6px7CE5DDyOr%2FfJH"
```

而新增，修改，删除 cookie 都是通过 document.cookie 来操作的，这就容易给人一个错觉，操作 cookie 要全量赋值。实则不然，每次 document.cookie=xxx 都是针对一个 cookie 的。所以，新增和修改一个 cookie 可以是：

```
JavaScript 复制代码
1 document.cookie = "test=1; path=/; domain=localhost; max-age=100; expire"
```

path 和 domain 限制了 cookie 的有效路径和域名。不过 path 限制是可以被绕过的，可以创建一个 iframe 打开特定 path，然后通过 iframe 的 document.cookie 读取到。保护 cookie 不被非法访问的唯一方法是将它放在另一个域名或子域名之下，利用同源策略保护其不被读取。

max-age 不支持低版本的 ie，所以和 expires 是结合起来用的，max-age 的优先级更高！expires 使用的是 GMT（格林威治）时间。

注意，secure 控制的是，仅在 https 协议中传输该 cookie。

删除一个 cookie 只要给它的 max-age 设置为 -1 即可。

```
document.cookie = 'test=1; max-age=-1'
```

cookie 的过期控制

cookie 可以通过 max-age 和 expires 控制过期时间，如果不设置过期时间，cookie 会在浏览器完全关闭时失效。

防止 cookie 被 javascript 脚本操作

可以在 Set-Cookie 时使用 HttpOnly 控制。一定程度上缓解 XSS 攻击

服务端 Set-Cookie

一个 Set-Cookie 也只能设置一个 cookie，如果需要设置多个，则返回多个 Set-Cookie 头。

Set-Cookie 和 document.cookie 操作类似，但是要注意两个选项，分别是 HttpOnly 和 SameSite，

HttpOnly 在上一小节说过了。SameSite 则是预防 CSRF 攻击的一种手段，允许服务器要求某个 cookie 在跨站请求时不会被发送。

- SameSite=Strict: 跨站请求时，不携带 cookie。
- SameSite=Lax: 宽松的，跨站请求时，也不会携带 cookie。但是从外部站点导航到本站点时会携带 cookie。Lax 是高版本 Chrome (80 版本以上) 的默认设置。

Lax 保证了这种场景可用：比如你在 github 已经登录过了，现在希望通过另外一个网站的链接跳转到 github，并且能保持 gitlab 的登录态，就需要用到 Lax。而 Strict 会阻止这一行为，这可能会要求你重新登录 github。

- SameSite=None: 不做限制，跨站请求时，会携带 cookie。使用 SameSite=None 时，需要配合 Secure 使用，也就是要求 Https 协议下才能使用 None。

跨站 cookie

恶意的跨站传递 cookie

在被恶意注入的情况下（比如 XSS），就可能导致本站点的 cookie 泄露。比如一个论坛留言板，被 XSS 注入了一段脚本，脚本会动态加载一张图片，图片的地址可能是黑客服务。

这样一来，只要用户打开留言板，自己的 cookie 信息就泄露了，就有可能被黑客冒充身份，造成损失。

业务需要的共享 cookie

为了满足类似于单点登录这样的功能，我们在统一认证平台 authgateway.myenterprise.com 登录后，希望公司内部另一个站点 other.myenterprise.com 也能共享这种登录状态。那么服务器端在 authgateway.myenterprise.com 登录成功后，应该 Set-Cookie 时将 domain 部分设置为 myenterprise.com，这样 myenterprise.com 的各个子域名才能共享登录状态。

但是这样也有风险，一旦某个子站被 XSS，就有可能全部被攻击（Cookie 作用域攻击）。最好启用全站 HTTPS+Secure，也必须适当缩小 cookie 的作用域（限制 domain 和 path）。

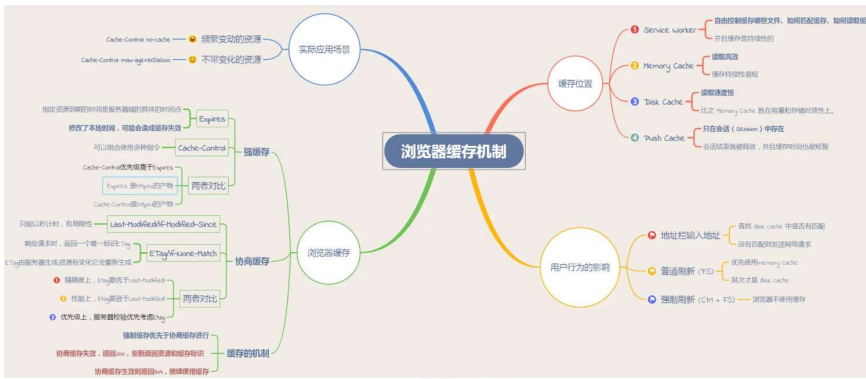
cookie 篡改

这个就很好理解了，document.cookie 是可以操作 cookie 的，这就留下了一些隐患，对于关键 cookie 信息，必须启用 HttpOnly 防护。

cookie 劫持

如果没有 HTTPS 证书的保障，cookie 也是明文传输，相当于裸奔。除了 XSS 攻击劫持 cookie，攻击者还可以通过中间人攻击获取 HTTP 报文中的

cookie。对于这些劫持情况，应该对 cookie 加密/签名或者采用 Https 防范！**强缓存和协商缓存**



说下 TCP 握手和挥手过程

三次握手，五次挥手，每一次的目的要知道

HTTP 1.1 的队头阻塞问题编解码/加密 base64

encodeURIComponent / decodeURIComponent 对称加密

非对称加密数字签名/验签

工程能力

模块化介绍？CMD 和 ESM 的区别：

mjs 是什么？

什么是 Tree Shaking？其基本原理是什么？按需加载的实现原理

Webpack 的一些细节代码规范细节

Git 工作流细节 npm 相关

Web 安全

从攻击者和防御者的角度来看 Web 安全，而不是背书。

跨域

跨域主要是针对 Javascript 的限制，防止恶意的脚本窃取/破坏/滥用用户数据。

什么是跨域？

同源的判断三要素是 协议、主机（域名/IP）、端口，只要三者中任何一个不同，就会发生跨域。

注意，即使是二级域名与子域名之间，也存在跨域问题。

PS: IE 并未将端口纳入检测的范围。



跨域产生了哪些限制？有哪些疑惑点？

- 写操作或嵌入操作是允许的，读操作是禁止的：很奇怪吧？你可以在在在网站中链接到其他网站，或者提交跨域的表单；你也可以嵌入跨域的图片，视频等媒体资源，甚至嵌入跨域的
iframe（前提是 X-Frame-Options 不被设置为 Deny 等值）；但是你不能通过 canvas 跨域图片的文件细节。
- 为什么 form 不被同源策略限制：form 提交一个 action 后，其处理过程完全交由浏览器和目标服务器，是不能指定脚本回调的。而通过 Ajax 发出的请求是可以获取到响应内容的，这就有极大风险，所以浏览器同源策略限制了跨域 Ajax 和 Fetch。但是，这也不是说 form 就是安全的，诱骗表单或 XSS 注入的恶意表单也是危险性很高的。
- canvas 操作图片的跨域：
- XMLHttpRequest: 无法跨域请求，这个是遇到最多的。
- preflight（预检）：
- Cookie: 出于安全考虑，无法跨域读取或设置 cookie（无论是从服务端还是客户端）。只能通过将 cookie 的 domain 设置为一个父域名来达到父域名下站点共享该 cookie 的目的。

如何解决跨域问题？

- jsonp
- CORS
- nginx 反向代理
- nodejs 服务代理

同源和同站

Same Origin 和 Same Site 的判断依据。TLD 和 eTLD。

点击劫持 Click Jacking

黑客通过一个 iframe 嵌套了目标网站，并通过一些 opacity 等技巧，在最上层覆盖一些恶意网站的链接，诱骗用户点击。用户以为自己访问的是百度，其实是一个假百度（黑客的域名进去的，但是用户没注意域名，只看到界面是百度）。

解决方案：HTTP headers 设置 X-FRAME-OPTIONS，用 deny 或者 sameorigin 是比较安全的，或者通过 allow-from 指定 URI 白名单。

中间人攻击

Man-in-the-middle attack <<https://developer.mozilla.org/en-US/docs/Glossary/MitM>>，通信过程被黑客劫持，就像邮递员可能篡改邮件一样。

预防中间人攻击：使用 HTTPS；用户认证信息和 IP 做绑定。

什么是 XSS 攻击？防御手段

XSS 是 Crossing-Site-Scripting，也就是跨站脚本攻击，指的是黑客恶意在目标网站植入一段脚本，用于窃取用户信息和伪装成用户，甚至执行其他恶意为。常发生于未对用户输入做校验和未对 innerHTML 做过滤的情况。

XSS 防御手段

- Set-Cookie 增加 HttpOnly 和 Secure 设置

什么是 CSRF 攻击？防御手段

CSRF 防御手段

- Set-Cookie 增加 SameSite 约束
- 不使用Cookie 这种自动携带的身份验证手段，改用JWT 等自定义 Request Header 的方案。
- CSRF Token
- 关键业务用验证码等方式强制校验。



Typescript

private, protected, public type, interface 类型和接口联合类型，交

叉类型

泛型

keyof, typeof 的使用

Utility Type 的实现原理

计算机网络

CDN，回源

CDN 是内容分发网络，源站提供内容，CDN 节点进行内容分发，用户可以就近获取所需内容，降低网络拥塞，提高用户访问响应速度和命中率。

回源，当有用户访问某一个 URL 的时候，如果被解析到的那个 CDN 节点没有缓存响应内容，或者是缓存已经到期，就会回源站去获取。如果没有人访问，那么 CDN 节点不会主动去源站拿的。

回源的策略间接决定了缓存的命中率，频繁回源会对源站产生较大的负担。

算法修炼

常见数据结构

- 数组（线性表）
- 链表（单向链表，双向链表，循环链表）
- 栈
- 队列，优先队列
- 哈希表
- 堆：最大堆，最小堆
- 树：二叉树，红黑树
- 图
- 字符串：前缀树（字典树） / 后缀树

时间/空间复杂度怎么算？

时间复杂度：取最坏情况下的算法执行步骤，忽略常数项，低次项。

空间复杂度：递归深度 N * 每次递归所要创建的变量数。



技巧

- 链表转数组：迭代就行了
- 反转链表：链表先转栈是一种办法，但是太慢了；反转其实是改变箭头方向，可以利用prev 存储前一个节点，依次迭代修改 next。
- 双指针遍历：指针相遇即结束
- 快慢指针：判断环
- dummy header（虚拟节点）：用于生成链表
- hashtable：值作为 key，值出现的次数作为 value

排序 排序算法的稳定性？

排序算法	时间复杂度（平均）	空间复杂度	稳定性
冒泡排序	$O(n^2)$	$O(1)$	稳定
快速排序	$O(n \log n)$	$O(\log n)$	不稳定
插入排序	$O(n^2)$	$O(1)$	稳定
选择排序	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n \log n)$	$O(1)$	不稳定
归并排序	$O(n \log n)$	$O(n)$	稳定

	时间复杂度	空间复杂度	稳定性
快速排序	$O(n \log n)$	$O(\log n)$	不稳定
插入排序	$O(n^2)$	$O(1)$	稳定
选择排序	$O(n^2)$	$O(1)$	不稳定
归并排序			

选择排序快速排序动态规划滑动窗口

LRU 算法

综合题

项目亮点总结